

- 17.23 Max Black Square:** Imagine you have a square matrix, where each cell (pixel) is either black or white. Design an algorithm to find the maximum subsquare such that all four borders are filled with black pixels.

Hints: #684, #695, #705, #714, #721, #736

pg 608

- 17.24 Max Submatrix:** Given an $N \times N$ matrix of positive and negative integers, write code to find the submatrix with the largest possible sum.

Hints: #469, #511, #525, #539, #565, #581, #595, #615, #621

pg 611

- 17.25 Word Rectangle:** Given a list of millions of words, design an algorithm to create the largest possible rectangle of letters such that every row forms a word (reading left to right) and every column forms a word (reading top to bottom). The words need not be chosen consecutively from the list, but all rows must be the same length and all columns must be the same height.

Hints: #477, #500, #748

pg 615

- 17.26 Sparse Similarity:** The similarity of two documents (each with distinct words) is defined to be the size of the intersection divided by the size of the union. For example, if the documents consist of integers, the similarity of {1, 5, 3} and {1, 7, 2, 3} is 0.4, because the intersection has size 2 and the union has size 5.

We have a long list of documents (with distinct values and each with an associated ID) where the similarity is believed to be "sparse." That is, any two arbitrarily selected documents are very likely to have similarity 0. Design an algorithm that returns a list of pairs of document IDs and the associated similarity.

Print only the pairs with similarity greater than 0. Empty documents should not be printed at all. For simplicity, you may assume each document is represented as an array of distinct integers.

EXAMPLE

Input:

```
13: {14, 15, 100, 9, 3}
16: {32, 1, 9, 3, 5}
19: {15, 29, 2, 6, 8, 7}
24: {7, 10}
```

Output:

```
ID1, ID2 : SIMILARITY
13, 19   : 0.1
13, 16   : 0.25
19, 24   : 0.14285714285714285
```

Hints: #484, #498, #510, #518, #534, #547, #555, #561, #569, #577, #584, #603, #611, #636

pg 620

X

Solutions

Join us at www.CrackingTheCodingInterview.com to download the complete solutions, contribute or view solutions in other programming languages, discuss problems from this book with other readers, ask questions, report issues, view this book's errata, and seek additional advice.

1

Solutions to Arrays and Strings

1.1 Is Unique: Implement an algorithm to determine if a string has all unique characters. What if you cannot use additional data structures?

pg 90

SOLUTION

You should first ask your interviewer if the string is an ASCII string or a Unicode string. Asking this question will show an eye for detail and a solid foundation in computer science. We'll assume for simplicity the character set is ASCII. If this assumption is not valid, we would need to increase the storage size.

One solution is to create an array of boolean values, where the flag at index *i* indicates whether character *i* in the alphabet is contained in the string. The second time you see this character you can immediately return false.

We can also immediately return false if the string length exceeds the number of unique characters in the alphabet. After all, you can't form a string of 280 unique characters out of a 128-character alphabet.

It's also okay to assume 256 characters. This would be the case in extended ASCII. You should clarify your assumptions with your interviewer.

The code below implements this algorithm.

```
1  boolean isUniqueChars(String str) {
2      if (str.length() > 128) return false;
3
4      boolean[] char_set = new boolean[128];
5      for (int i = 0; i < str.length(); i++) {
6          int val = str.charAt(i);
7          if (char_set[val]) { // Already found this char in string
8              return false;
9          }
10         char_set[val] = true;
11     }
12     return true;
13 }
```

The time complexity for this code is $O(n)$, where *n* is the length of the string. The space complexity is $O(1)$. (You could also argue the time complexity is $O(1)$, since the for loop will never iterate through more than 128 characters.) If you didn't want to assume the character set is fixed, you could express the complexity as $O(c)$ space and $O(\min(c, n))$ or $O(c)$ time, where *c* is the size of the character set.

We can reduce our space usage by a factor of eight by using a bit vector. We will assume, in the below code, that the string only uses the lowercase letters a through z. This will allow us to use just a single `int`.

```

1  boolean isUniqueChars(String str) {
2      int checker = 0;
3      for (int i = 0; i < str.length(); i++) {
4          int val = str.charAt(i) - 'a';
5          if ((checker & (1 << val)) > 0) {
6              return false;
7          }
8          checker |= (1 << val);
9      }
10     return true;
11 }

```

If we can't use additional data structures, we can do the following:

1. Compare every character of the string to every other character of the string. This will take $O(n^2)$ time and $O(1)$ space.
2. If we are allowed to modify the input string, we could sort the string in $O(n \log(n))$ time and then linearly check the string for neighboring characters that are identical. Careful, though: many sorting algorithms take up extra space.

These solutions are not as optimal in some respects, but might be better depending on the constraints of the problem.

1.2 Check Permutation: Given two strings, write a method to decide if one is a permutation of the other.

pg 90

SOLUTION

Like in many questions, we should confirm some details with our interviewer. We should understand if the permutation comparison is case sensitive. That is: is God a permutation of dog? Additionally, we should ask if whitespace is significant. We will assume for this problem that the comparison is case sensitive and whitespace is significant. So, "god" is different from "dog".

Observe first that strings of different lengths cannot be permutations of each other. There are two easy ways to solve this problem, both of which use this optimization.

Solution #1: Sort the strings.

If two strings are permutations, then we know they have the same characters, but in different orders. Therefore, sorting the strings will put the characters from two permutations in the same order. We just need to compare the sorted versions of the strings.

```

1  String sort(String s) {
2      char[] content = s.toCharArray();
3      java.util.Arrays.sort(content);
4      return new String(content);
5  }
6
7  boolean permutation(String s, String t) {
8      if (s.length() != t.length()) {
9          return false;
10     }

```

```
11     return sort(s).equals(sort(t));
12 }
```

Though this algorithm is not as optimal in some senses, it may be preferable in one sense: It's clean, simple and easy to understand. In a practical sense, this may very well be a superior way to implement the problem.

However, if efficiency is very important, we can implement it a different way.

Solution #2: Check if the two strings have identical character counts.

We can also use the definition of a permutation—two words with the same character counts—to implement this algorithm. We simply iterate through this code, counting how many times each character appears. Then, afterwards, we compare the two arrays.

```
1  boolean permutation(String s, String t) {
2      if (s.length() != t.length()) {
3          return false;
4      }
5
6      int[] letters = new int[128]; // Assumption
7
8      char[] s_array = s.toCharArray();
9      for (char c : s_array) { // count number of each char in s.
10         letters[c]++;
11     }
12
13     for (int i = 0; i < t.length(); i++) {
14         int c = (int) t.charAt(i);
15         letters[c]--;
16         if (letters[c] < 0) {
17             return false;
18         }
19     }
20
21     return true;
22 }
```

Note the assumption on line 6. In your interview, you should always check with your interviewer about the size of the character set. We assumed that the character set was ASCII.

1.3 URLify: Write a method to replace all spaces in a string with '%20'. You may assume that the string has sufficient space at the end to hold the additional characters, and that you are given the "true" length of the string. (Note: if implementing in Java, please use a character array so that you can perform this operation in place.)

EXAMPLE

Input: "Mr John Smith ", 13

Output: "Mr%20John%20Smith"

pg 90

SOLUTION

A common approach in string manipulation problems is to edit the string starting from the end and working backwards. This is useful because we have an extra buffer at the end, which allows us to change characters without worrying about what we're overwriting.

We will use this approach in this problem. The algorithm employs a two-scan approach. In the first scan, we count the number of spaces. By tripling this number, we can compute how many extra characters we will have in the final string. In the second pass, which is done in reverse order, we actually edit the string. When we see a space, we replace it with %20. If there is no space, then we copy the original character.

The code below implements this algorithm.

```

1 void replaceSpaces(char[] str, int trueLength) {
2     int spaceCount = 0, index, i = 0;
3     for (i = 0; i < trueLength; i++) {
4         if (str[i] == ' ') {
5             spaceCount++;
6         }
7     }
8     index = trueLength + spaceCount * 2;
9     if (trueLength < str.length) str[trueLength] = '\0'; // End array
10    for (i = trueLength - 1; i >= 0; i--) {
11        if (str[i] == ' ') {
12            str[index - 1] = '0';
13            str[index - 2] = '2';
14            str[index - 3] = '%';
15            index = index - 3;
16        } else {
17            str[index - 1] = str[i];
18            index--;
19        }
20    }
21 }

```

We have implemented this problem using character arrays, because Java strings are immutable. If we used strings directly, the function would have to return a new copy of the string, but it would allow us to implement this in just one pass.

1.4 Palindrome Permutation: Given a string, write a function to check if it is a permutation of a palindrome. A palindrome is a word or phrase that is the same forwards and backwards. A permutation is a rearrangement of letters. The palindrome does not need to be limited to just dictionary words.

EXAMPLE

Input: Tact Coa

Output: True (permutations: "taco cat", "atco cta", etc.)

pg 91

SOLUTION

This is a question where it helps to figure out what it means for a string to be a permutation of a palindrome. This is like asking what the "defining features" of such a string would be.

A palindrome is a string that is the same forwards and backwards. Therefore, to decide if a string is a permutation of a palindrome, we need to know if it can be written such that it's the same forwards and backwards.

What does it take to be able to write a set of characters the same way forwards and backwards? We need to have an even number of almost all characters, so that half can be on one side and half can be on the other side. At most one character (the middle character) can have an odd count.

For example, we know tact coapapa is a permutation of a palindrome because it has two Ts, four As, two

Cs, two Ps, and one O. That O would be the center of all possible palindromes.

To be more precise, strings with even length (after removing all non-letter characters) must have all even counts of characters. Strings of an odd length must have exactly one character with an odd count. Of course, an “even” string can’t have an odd number of exactly one character, otherwise it wouldn’t be an even-length string (an odd number + many even numbers = an odd number). Likewise, a string with odd length can’t have all characters with even counts (sum of evens is even). It’s therefore sufficient to say that, to be a permutation of a palindrome, a string can have no more than one character that is odd. This will cover both the odd and the even cases.

This leads us to our first algorithm.

Solution #1

Implementing this algorithm is fairly straightforward. We use a hash table to count how many times each character appears. Then, we iterate through the hash table and ensure that no more than one character has an odd count.

```
1  boolean isPermutationOfPalindrome(String phrase) {
2      int[] table = buildCharFrequencyTable(phrase);
3      return checkMaxOneOdd(table);
4  }
5
6  /* Check that no more than one character has an odd count. */
7  boolean checkMaxOneOdd(int[] table) {
8      boolean foundOdd = false;
9      for (int count : table) {
10         if (count % 2 == 1) {
11             if (foundOdd) {
12                 return false;
13             }
14             foundOdd = true;
15         }
16     }
17     return true;
18 }
19
20 /* Map each character to a number. a -> 0, b -> 1, c -> 2, etc.
21 * This is case insensitive. Non-letter characters map to -1. */
22 int getCharNumber(Character c) {
23     int a = Character.getNumericValue('a');
24     int z = Character.getNumericValue('z');
25     int val = Character.getNumericValue(c);
26     if (a <= val && val <= z) {
27         return val - a;
28     }
29     return -1;
30 }
31
32 /* Count how many times each character appears. */
33 int[] buildCharFrequencyTable(String phrase) {
34     int[] table = new int[Character.getNumericValue('z') -
35                          Character.getNumericValue('a') + 1];
36     for (char c : phrase.toCharArray()) {
37         int x = getCharNumber(c);
```

```

38     if (x != -1) {
39         table[x]++;
40     }
41 }
42 return table;
43 }

```

This algorithm takes $O(N)$ time, where N is the length of the string.

Solution #2

We can't optimize the big O time here since any algorithm will always have to look through the entire string. However, we can make some smaller incremental improvements. Because this is a relatively simple problem, it can be worthwhile to discuss some small optimizations or at least some tweaks.

Instead of checking the number of odd counts at the end, we can check as we go along. Then, as soon as we get to the end, we have our answer.

```

1  boolean isPermutationOfPalindrome(String phrase) {
2      int countOdd = 0;
3      int[] table = new int[Character.getNumericValue('z') -
4                          Character.getNumericValue('a') + 1];
5      for (char c : phrase.toCharArray()) {
6          int x = getCharNumber(c);
7          if (x != -1) {
8              table[x]++;
9              if (table[x] % 2 == 1) {
10                 countOdd++;
11             } else {
12                 countOdd--;
13             }
14         }
15     }
16     return countOdd <= 1;
17 }

```

It's important to be very clear here that this is not necessarily more optimal. It has the same big O time and might even be slightly slower. We have eliminated a final iteration through the hash table, but now we have to run a few extra lines of code for each character in the string.

You should discuss this with your interviewer as an alternate, but not necessarily more optimal, solution.

Solution #3

If you think more deeply about this problem, you might notice that we don't actually need to know the counts. We just need to know if the count is even or odd. Think about flipping a light on/off (that is initially off). If the light winds up in the off state, we don't know how many times we flipped it, but we do know it was an even count.

Given this, we can use a single integer (as a bit vector). When we see a letter, we map it to an integer between 0 and 26 (assuming an English alphabet). Then we toggle the bit at that value. At the end of the iteration, we check that at most one bit in the integer is set to 1.

We can easily check that no bits in the integer are 1: just compare the integer to 0. There is actually a very elegant way to check that an integer has exactly one bit set to 1.

Picture an integer like 00010000. We could of course shift the integer repeatedly to check that there's only a single 1. Alternatively, if we subtract 1 from the number, we'll get 00001111. What's notable about this

is that there is no overlap between the numbers (as opposed to say 00101000, which, when we subtract 1 from, we get 00100111.) So, we can check to see that a number has exactly one 1 because if we subtract 1 from it and then AND it with the new number, we should get 0.

$$\begin{aligned}00010000 - 1 &= 00001111 \\ 00010000 \& 00001111 &= 0\end{aligned}$$

This leads us to our final implementation.

```
1  boolean isPermutationOfPalindrome(String phrase) {
2      int bitVector = createBitVector(phrase);
3      return bitVector == 0 || checkExactlyOneBitSet(bitVector);
4  }
5
6  /* Create a bit vector for the string. For each letter with value i, toggle the
7   * ith bit. */
8  int createBitVector(String phrase) {
9      int bitVector = 0;
10     for (char c : phrase.toCharArray()) {
11         int x = getCharNumber(c);
12         bitVector = toggle(bitVector, x);
13     }
14     return bitVector;
15 }
16
17 /* Toggle the ith bit in the integer. */
18 int toggle(int bitVector, int index) {
19     if (index < 0) return bitVector;
20
21     int mask = 1 << index;
22     if ((bitVector & mask) == 0) {
23         bitVector |= mask;
24     } else {
25         bitVector &= ~mask;
26     }
27     return bitVector;
28 }
29
30 /* Check that exactly one bit is set by subtracting one from the integer and
31  * ANDing it with the original integer. */
32 boolean checkExactlyOneBitSet(int bitVector) {
33     return (bitVector & (bitVector - 1)) == 0;
34 }
```

Like the other solutions, this is $O(N)$.

It's interesting to note a solution that we did not explore. We avoided solutions along the lines of "create all possible permutations and check if they are palindromes." While such a solution would work, it's entirely infeasible in the real world. Generating all permutations requires factorial time (which is actually worse than exponential time), and it is essentially infeasible to perform on strings longer than about 10–15 characters.

I mention this (impractical) solution because a lot of candidates hear a problem like this and say, "In order to check if A is in group B, I must know everything that is in B and then check if one of the items equals A." That's not always the case, and this problem is a simple demonstration of it. You don't need to generate all permutations in order to check if one is a palindrome.

- 1.5 One Away:** There are three types of edits that can be performed on strings: insert a character, remove a character, or replace a character. Given two strings, write a function to check if they are one edit (or zero edits) away.

EXAMPLE

```
pale, ple -> true
pales, pale -> true
pale, bale -> true
pale, bae -> false
```

pg 91

SOLUTION

There is a “brute force” algorithm to do this. We could check all possible strings that are one edit away by testing the removal of each character (and comparing), testing the replacement of each character (and comparing), and then testing the insertion of each possible character (and comparing).

That would be too slow, so let’s not bother with implementing it.

This is one of those problems where it’s helpful to think about the “meaning” of each of these operations. What does it mean for two strings to be one insertion, replacement, or removal away from each other?

- **Replacement:** Consider two strings, such as *bale* and *pale*, that are one replacement away. Yes, that does mean that you could replace a character in *bale* to make *pale*. But more precisely, it means that they are different only in one place.
- **Insertion:** The strings *apple* and *aple* are one insertion away. This means that if you compared the strings, they would be identical—except for a shift at some point in the strings.
- **Removal:** The strings *apple* and *aple* are also one removal away, since removal is just the inverse of insertion.

We can go ahead and implement this algorithm now. We’ll merge the insertion and removal check into one step, and check the replacement step separately.

Observe that you don’t need to check the strings for insertion, removal, and replacement edits. The lengths of the strings will indicate which of these you need to check.

```
1  boolean oneEditAway(String first, String second) {
2      if (first.length() == second.length()) {
3          return oneEditReplace(first, second);
4      } else if (first.length() + 1 == second.length()) {
5          return oneEditInsert(first, second);
6      } else if (first.length() - 1 == second.length()) {
7          return oneEditInsert(second, first);
8      }
9      return false;
10 }
11
12 boolean oneEditReplace(String s1, String s2) {
13     boolean foundDifference = false;
14     for (int i = 0; i < s1.length(); i++) {
15         if (s1.charAt(i) != s2.charAt(i)) {
16             if (foundDifference) {
17                 return false;
18             }
19         }
20     }
```

```
20         foundDifference = true;
21     }
22 }
23 return true;
24 }
25
26 /* Check if you can insert a character into s1 to make s2. */
27 boolean oneEditInsert(String s1, String s2) {
28     int index1 = 0;
29     int index2 = 0;
30     while (index2 < s2.length() && index1 < s1.length()) {
31         if (s1.charAt(index1) != s2.charAt(index2)) {
32             if (index1 != index2) {
33                 return false;
34             }
35             index2++;
36         } else {
37             index1++;
38             index2++;
39         }
40     }
41     return true;
42 }
```

This algorithm (and almost any reasonable algorithm) takes $O(n)$ time, where n is the length of the shorter string.

Why is the runtime dictated by the shorter string instead of the longer string? If the strings are the same length (plus or minus one character), then it doesn't matter whether we use the longer string or the shorter string to define the runtime. If the strings are very different lengths, then the algorithm will terminate in $O(1)$ time. One really, really long string therefore won't significantly extend the runtime. It increases the runtime only if both strings are long.

We might notice that the code for `oneEditReplace` is very similar to that for `oneEditInsert`. We can merge them into one method.

To do this, observe that both methods follow similar logic: compare each character and ensure that the strings are only different by one. The methods vary in how they handle that difference. The method `oneEditReplace` does nothing other than flag the difference, whereas `oneEditInsert` increments the pointer to the longer string. We can handle both of these in the same method.

```
1  boolean oneEditAway(String first, String second) {
2      /* Length checks. */
3      if (Math.abs(first.length() - second.length()) > 1) {
4          return false;
5      }
6
7      /* Get shorter and longer string.*/
8      String s1 = first.length() < second.length() ? first : second;
9      String s2 = first.length() < second.length() ? second : first;
10
11     int index1 = 0;
12     int index2 = 0;
13     boolean foundDifference = false;
14     while (index2 < s2.length() && index1 < s1.length()) {
15         if (s1.charAt(index1) != s2.charAt(index2)) {
```

```

16      /* Ensure that this is the first difference found.*/
17      if (foundDifference) return false;
18      foundDifference = true;
19
20      if (s1.length() == s2.length()) { // On replace, move shorter pointer
21          index1++;
22      }
23      } else {
24          index1++; // If matching, move shorter pointer
25      }
26      index2++; // Always move pointer for longer string
27  }
28  return true;
29  }

```

Some people might argue the first approach is better, as it is clearer and easier to follow. Others, however, will argue that the second approach is better, since it's more compact and doesn't duplicate code (which can facilitate maintainability).

You don't necessarily need to "pick a side." You can discuss the tradeoffs with your interviewer.

1.6 String Compression: Implement a method to perform basic string compression using the counts of repeated characters. For example, the string `aabcccccaaa` would become `a2b1c5a3`. If the "compressed" string would not become smaller than the original string, your method should return the original string. You can assume the string has only uppercase and lowercase letters (a - z).

pg 91

SOLUTION

At first glance, implementing this method seems fairly straightforward, but perhaps a bit tedious. We iterate through the string, copying characters to a new string and counting the repeats. At each iteration, check if the current character is the same as the next character. If not, add its compressed version to the result.

How hard could it be?

```

1  String compressBad(String str) {
2      String compressedString = "";
3      int countConsecutive = 0;
4      for (int i = 0; i < str.length(); i++) {
5          countConsecutive++;
6
7          /* If next character is different than current, append this char to result.*/
8          if (i + 1 >= str.length() || str.charAt(i) != str.charAt(i + 1)) {
9              compressedString += "" + str.charAt(i) + countConsecutive;
10             countConsecutive = 0;
11         }
12     }
13     return compressedString.length() < str.length() ? compressedString : str;
14 }

```

This works. Is it efficient, though? Take a look at the runtime of this code.

The runtime is $O(p + k^2)$, where p is the size of the original string and k is the number of character sequences. For example, if the string is `aabccdeaaa`, then there are six character sequences. It's slow because string concatenation operates in $O(n^2)$ time (see `StringBuilder` on pg 89).

We can fix this by using a `StringBuilder`.


```
1 String compress(String str) {
2     StringBuilder compressed = new StringBuilder();
3     int countConsecutive = 0;
4     for (int i = 0; i < str.length(); i++) {
5         countConsecutive++;
6
7         /* If next character is different than current, append this char to result.*/
8         if (i + 1 >= str.length() || str.charAt(i) != str.charAt(i + 1)) {
9             compressed.append(str.charAt(i));
10            compressed.append(countConsecutive);
11            countConsecutive = 0;
12        }
13    }
14    return compressed.length() < str.length() ? compressed.toString() : str;
15 }
```

Both of these solutions create the compressed string first and then return the shorter of the input string and the compressed string.

Instead, we can check in advance. This will be more optimal in cases where we don't have a large number of repeating characters. It will avoid us having to create a string that we never use. The downside of this is that it causes a second loop through the characters and also adds nearly duplicated code.

```
1 String compress(String str) {
2     /* Check final length and return input string if it would be longer. */
3     int finalLength = countCompression(str);
4     if (finalLength >= str.length()) return str;
5
6     StringBuilder compressed = new StringBuilder(finalLength); // initial capacity
7     int countConsecutive = 0;
8     for (int i = 0; i < str.length(); i++) {
9         countConsecutive++;
10
11        /* If next character is different than current, append this char to result.*/
12        if (i + 1 >= str.length() || str.charAt(i) != str.charAt(i + 1)) {
13            compressed.append(str.charAt(i));
14            compressed.append(countConsecutive);
15            countConsecutive = 0;
16        }
17    }
18    return compressed.toString();
19 }
20
21 int countCompression(String str) {
22     int compressedLength = 0;
23     int countConsecutive = 0;
24     for (int i = 0; i < str.length(); i++) {
25         countConsecutive++;
26
27        /* If next character is different than current, increase the length.*/
28        if (i + 1 >= str.length() || str.charAt(i) != str.charAt(i + 1)) {
29            compressedLength += 1 + String.valueOf(countConsecutive).length();
30            countConsecutive = 0;
31        }
32    }
33    return compressedLength;
34 }
```

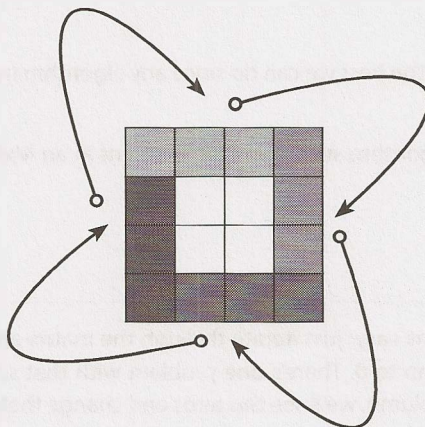
One other benefit of this approach is that we can initialize `StringBuilder` to its necessary capacity up-front. Without this, `StringBuilder` will (behind the scenes) need to double its capacity every time it hits capacity. The capacity could be double what we ultimately need.

1.7 Rotate Matrix: Given an image represented by an $N \times N$ matrix, where each pixel in the image is 4 bytes, write a method to rotate the image by 90 degrees. Can you do this in place?

pg 91

SOLUTION

Because we're rotating the matrix by 90 degrees, the easiest way to do this is to implement the rotation in layers. We perform a circular rotation on each layer, moving the top edge to the right edge, the right edge to the bottom edge, the bottom edge to the left edge, and the left edge to the top edge.



How do we perform this four-way edge swap? One option is to copy the top edge to an array, and then move the left to the top, the bottom to the left, and so on. This requires $O(N)$ memory, which is actually unnecessary.

A better way to do this is to implement the swap index by index. In this case, we do the following:

```
1 for i = 0 to n
2     temp = top[i];
3     top[i] = left[i]
4     left[i] = bottom[i]
5     bottom[i] = right[i]
6     right[i] = temp
```

We perform such a swap on each layer, starting from the outermost layer and working our way inwards. (Alternatively, we could start from the inner layer and work outwards.)

The code for this algorithm is below.

```
1 boolean rotate(int[][] matrix) {
2     if (matrix.length == 0 || matrix.length != matrix[0].length) return false;
3     int n = matrix.length;
4     for (int layer = 0; layer < n / 2; layer++) {
5         int first = layer;
6         int last = n - 1 - layer;
7         for (int i = first; i < last; i++) {
8             int offset = i - first;
```

```
9         int top = matrix[first][i]; // save top
10
11         // left -> top
12         matrix[first][i] = matrix[last-offset][first];
13
14         // bottom -> left
15         matrix[last-offset][first] = matrix[last][last - offset];
16
17         // right -> bottom
18         matrix[last][last - offset] = matrix[i][last];
19
20         // top -> right
21         matrix[i][last] = top; // right <- saved top
22     }
23 }
24 return true;
25 }
```

This algorithm is $O(N^2)$, which is the best we can do since any algorithm must touch all N^2 elements.

1.8 Zero Matrix: Write an algorithm such that if an element in an $M \times N$ matrix is 0, its entire row and column are set to 0.

pg 91

SOLUTION

At first glance, this problem seems easy: just iterate through the matrix and every time we see a cell with value zero, set its row and column to 0. There's one problem with that solution though: when we come across other cells in that row or column, we'll see the zeros and change their row and column to zero. Pretty soon, our entire matrix will be set to zeros.

One way around this is to keep a second matrix which flags the zero locations. We would then do a second pass through the matrix to set the zeros. This would take $O(MN)$ space.

Do we really need $O(MN)$ space? No. Since we're going to set the entire row and column to zero, we don't need to track that it was exactly `cell[2][4]` (row 2, column 4). We only need to know that row 2 has a zero somewhere, and column 4 has a zero somewhere. We'll set the entire row and column to zero anyway, so why would we care to keep track of the exact location of the zero?

The code below implements this algorithm. We use two arrays to keep track of all the rows with zeros and all the columns with zeros. We then nullify rows and columns based on the values in these arrays.

```
1 void setZeros(int[][] matrix) {
2     boolean[] row = new boolean[matrix.length];
3     boolean[] column = new boolean[matrix[0].length];
4
5     // Store the row and column index with value 0
6     for (int i = 0; i < matrix.length; i++) {
7         for (int j = 0; j < matrix[0].length; j++) {
8             if (matrix[i][j] == 0) {
9                 row[i] = true;
10                column[j] = true;
11            }
12        }
13    }
14 }
```

```

15 // Nullify rows
16 for (int i = 0; i < row.length; i++) {
17     if (row[i]) nullifyRow(matrix, i);
18 }
19
20 // Nullify columns
21 for (int j = 0; j < column.length; j++) {
22     if (column[j]) nullifyColumn(matrix, j);
23 }
24 }
25
26 void nullifyRow(int[][] matrix, int row) {
27     for (int j = 0; j < matrix[0].length; j++) {
28         matrix[row][j] = 0;
29     }
30 }
31
32 void nullifyColumn(int[][] matrix, int col) {
33     for (int i = 0; i < matrix.length; i++) {
34         matrix[i][col] = 0;
35     }
36 }

```

To make this somewhat more space efficient, we could use a bit vector instead of a boolean array. It would still be $O(N)$ space.

We can reduce the space to $O(1)$ by using the first row as a replacement for the row array and the first column as a replacement for the column array. This works as follows:

1. Check if the first row and first column have any zeros, and set variables `rowHasZero` and `columnHasZero`. (We'll nullify the first row and first column later, if necessary.)
2. Iterate through the rest of the matrix, setting `matrix[i][0]` and `matrix[0][j]` to zero whenever there's a zero in `matrix[i][j]`.
3. Iterate through rest of matrix, nullifying row `i` if there's a zero in `matrix[i][0]`.
4. Iterate through rest of matrix, nullifying column `j` if there's a zero in `matrix[0][j]`.
5. Nullify the first row and first column, if necessary (based on values from Step 1).

This code is below:

```

1 void setZeros(int[][] matrix) {
2     boolean rowHasZero = false;
3     boolean colHasZero = false;
4
5     // Check if first row has a zero
6     for (int j = 0; j < matrix[0].length; j++) {
7         if (matrix[0][j] == 0) {
8             rowHasZero = true;
9             break;
10        }
11    }
12
13    // Check if first column has a zero
14    for (int i = 0; i < matrix.length; i++) {
15        if (matrix[i][0] == 0) {
16            colHasZero = true;
17            break;

```



```
18     }
19 }
20
21 // Check for zeros in the rest of the array
22 for (int i = 1; i < matrix.length; i++) {
23     for (int j = 1; j < matrix[0].length; j++) {
24         if (matrix[i][j] == 0) {
25             matrix[i][0] = 0;
26             matrix[0][j] = 0;
27         }
28     }
29 }
30
31 // Nullify rows based on values in first column
32 for (int i = 1; i < matrix.length; i++) {
33     if (matrix[i][0] == 0) {
34         nullifyRow(matrix, i);
35     }
36 }
37
38 // Nullify columns based on values in first row
39 for (int j = 1; j < matrix[0].length; j++) {
40     if (matrix[0][j] == 0) {
41         nullifyColumn(matrix, j);
42     }
43 }
44
45 // Nullify first row
46 if (rowHasZero) {
47     nullifyRow(matrix, 0);
48 }
49
50 // Nullify first column
51 if (colHasZero) {
52     nullifyColumn(matrix, 0);
53 }
54 }
```

This code has a lot of “do this for the rows, then the equivalent action for the column.” In an interview, you could abbreviate this code by adding comments and TODOs that explain that the next chunk of code looks the same as the earlier code, but using rows. This would allow you to focus on the most important parts of the algorithm.

1.9 String Rotation: Assume you have a method `isSubstring` which checks if one word is a substring of another. Given two strings, `s1` and `s2`, write code to check if `s2` is a rotation of `s1` using only one call to `isSubstring` (e.g., “waterbottle” is a rotation of “erbottlewat”).

pg 91

SOLUTION

If we imagine that `s2` is a rotation of `s1`, then we can ask what the rotation point is. For example, if you rotate `waterbottle` after `wat`, you get `erbottlewat`. In a rotation, we cut `s1` into two parts, `x` and `y`, and rearrange them to get `s2`.

```
s1 = xy = waterbottle
x = wat
```

```

y = erbottle
s2 = yx = erbottlewat

```

So, we need to check if there's a way to split $s1$ into x and y such that $xy = s1$ and $yx = s2$. Regardless of where the division between x and y is, we can see that yx will always be a substring of $xyxy$. That is, $s2$ will always be a substring of $s1s1$.

And this is precisely how we solve the problem: simply do `isSubstring(s1s1, s2)`.

The code below implements this algorithm.

```

1  boolean isRotation(String s1, String s2) {
2      int len = s1.length();
3      /* Check that s1 and s2 are equal length and not empty */
4      if (len == s2.length() && len > 0) {
5          /* Concatenate s1 and s1 within new buffer */
6          String s1s1 = s1 + s1;
7          return isSubstring(s1s1, s2);
8      }
9      return false;
10 }

```

The runtime of this varies based on the runtime of `isSubstring`. But if you assume that `isSubstring` runs in $O(A+B)$ time (on strings of length A and B), then the runtime of `isRotation` is $O(N)$.

2

Solutions to Linked Lists

2.1 Remove Dups: Write code to remove duplicates from an unsorted linked list.

FOLLOW UP

How would you solve this problem if a temporary buffer is not allowed?

pg 94

SOLUTION

In order to remove duplicates from a linked list, we need to be able to track duplicates. A simple hash table will work well here.

In the below solution, we simply iterate through the linked list, adding each element to a hash table. When we discover a duplicate element, we remove the element and continue iterating. We can do this all in one pass since we are using a linked list.

```
1 void deleteDups(LinkedListNode n) {
2     HashSet<Integer> set = new HashSet<Integer>();
3     LinkedListNode previous = null;
4     while (n != null) {
5         if (set.contains(n.data)) {
6             previous.next = n.next;
7         } else {
8             set.add(n.data);
9             previous = n;
10        }
11        n = n.next;
12    }
13 }
```

The above solution takes $O(N)$ time, where N is the number of elements in the linked list.

Follow Up: No Buffer Allowed

If we don't have a buffer, we can iterate with two pointers: `current` which iterates through the linked list, and `runner` which checks all subsequent nodes for duplicates.

```
1 void deleteDups(LinkedListNode head) {
2     LinkedListNode current = head;
3     while (current != null) {
4         /* Remove all future nodes that have the same value */
5         LinkedListNode runner = current;
6         while (runner.next != null) {
7             if (runner.next.data == current.data) {
```

```

8         runner.next = runner.next.next;
9     } else {
10         runner = runner.next;
11     }
12 }
13 current = current.next;
14 }
15 }

```

This code runs in $O(1)$ space, but $O(N^2)$ time.

2.2 Return Kth to Last: Implement an algorithm to find the kth to last element of a singly linked list.

pg 223

SOLUTION

We will approach this problem both recursively and non-recursively. Remember that recursive solutions are often cleaner but less optimal. For example, in this problem, the recursive implementation is about half the length of the iterative solution but also takes $O(n)$ space, where n is the number of elements in the linked list.

Note that for this solution, we have defined k such that passing in $k = 1$ would return the last element, $k = 2$ would return to the second to last element, and so on. It is equally acceptable to define k such that $k = 0$ would return the last element.

Solution #1: If linked list size is known

If the size of the linked list is known, then the kth to last element is the $(\text{length} - k)$ th element. We can just iterate through the linked list to find this element. Because this solution is so trivial, we can almost be sure that this is not what the interviewer intended.

Solution #2: Recursive

This algorithm recurses through the linked list. When it hits the end, the method passes back a counter set to 0. Each parent call adds 1 to this counter. When the counter equals k , we know we have reached the kth to last element of the linked list.

Implementing this is short and sweet—provided we have a way of “passing back” an integer value through the stack. Unfortunately, we can’t pass back a node and a counter using normal return statements. So how do we handle this?

Approach A: Don’t Return the Element.

One way to do this is to change the problem to simply printing the kth to last element. Then, we can pass back the value of the counter simply through return values.

```

1  int printKthToLast(LinkedListNode head, int k) {
2      if (head == null) {
3          return 0;
4      }
5      int index = printKthToLast(head.next, k) + 1;
6      if (index == k) {
7          System.out.println(k + "th to last node is " + head.data);
8      }
9      return index;
10 }

```


Of course, this is only a valid solution if the interviewer says it is valid.

Approach B: Use C++.

A second way to solve this is to use C++ and to pass values by reference. This allows us to return the node value, but also update the counter by passing a pointer to it.

```
1  node* nthToLast(node* head, int k, int& i) {
2      if (head == NULL) {
3          return NULL;
4      }
5      node* nd = nthToLast(head->next, k, i);
6      i = i + 1;
7      if (i == k) {
8          return head;
9      }
10     return nd;
11 }
12
13 node* nthToLast(node* head, int k) {
14     int i = 0;
15     return nthToLast(head, k, i);
16 }
```

Approach C: Create a Wrapper Class.

We described earlier that the issue was that we couldn't simultaneously return a counter and an index. If we wrap the counter value with simple class (or even a single element array), we can mimic passing by reference.

```
1  class Index {
2      public int value = 0;
3  }
4
5  LinkedListNode kthToLast(LinkedListNode head, int k) {
6      Index idx = new Index();
7      return kthToLast(head, k, idx);
8  }
9
10 LinkedListNode kthToLast(LinkedListNode head, int k, Index idx) {
11     if (head == null) {
12         return null;
13     }
14     LinkedListNode node = kthToLast(head.next, k, idx);
15     idx.value = idx.value + 1;
16     if (idx.value == k) {
17         return head;
18     }
19     return node;
20 }
```

Each of these recursive solutions takes $O(n)$ space due to the recursive calls.

There are a number of other solutions that we haven't addressed. We could store the counter in a static variable. Or, we could create a class that stores both the node and the counter, and return an instance of that class. Regardless of which solution we pick, we need a way to update both the node and the counter in a way that all levels of the recursive stack will see.

Solution #3: Iterative

A more optimal, but less straightforward, solution is to implement this iteratively. We can use two pointers, p1 and p2. We place them k nodes apart in the linked list by putting p2 at the beginning and moving p1 k nodes into the list. Then, when we move them at the same pace, p1 will hit the end of the linked list after $\text{LENGTH} - k$ steps. At that point, p2 will be $\text{LENGTH} - k$ nodes into the list, or k nodes from the end.

The code below implements this algorithm.

```

1  LinkedListNode nthToLast(LinkedListNode head, int k) {
2      LinkedListNode p1 = head;
3      LinkedListNode p2 = head;
4
5      /* Move p1 k nodes into the list.*/
6      for (int i = 0; i < k; i++) {
7          if (p1 == null) return null; // Out of bounds
8          p1 = p1.next;
9      }
10
11     /* Move them at the same pace. When p1 hits the end, p2 will be at the right
12      * element. */
13     while (p1 != null) {
14         p1 = p1.next;
15         p2 = p2.next;
16     }
17     return p2;
18 }
```

This algorithm takes $O(n)$ time and $O(1)$ space.

2.3 Delete Middle Node: Implement an algorithm to delete a node in the middle (i.e., any node but the first and last node, not necessarily the exact middle) of a singly linked list, given only access to that node.

EXAMPLE

Input: the node c from the linked list a->b->c->d->e->f

Result: nothing is returned, but the new linked list looks like a->b->d->e->f

pg 94

SOLUTION

In this problem, you are not given access to the head of the linked list. You only have access to that node. The solution is simply to copy the data from the next node over to the current node, and then to delete the next node.

The code below implements this algorithm.

```

1  boolean deleteNode(LinkedListNode n) {
2      if (n == null || n.next == null) {
3          return false; // Failure
4      }
5      LinkedListNode next = n.next;
6      n.data = next.data;
7      n.next = next.next;
8      return true;
9  }
```

Note that this problem cannot be solved if the node to be deleted is the last node in the linked list. That's okay—your interviewer wants you to point that out, and to discuss how to handle this case. You could, for example, consider marking the node as dummy.

2.4 Partition: Write code to partition a linked list around a value x , such that all nodes less than x come before all nodes greater than or equal to x . If x is contained within the list, the values of x only need to be after the elements less than x (see below). The partition element x can appear anywhere in the "right partition"; it does not need to appear between the left and right partitions.

EXAMPLE

Input: 3 -> 5 -> 8 -> 5 -> 10 -> 2 -> 1 [partition = 5]

Output: 3 -> 1 -> 2 -> 10 -> 5 -> 5 -> 8

pg 94

SOLUTION

If this were an array, we would need to be careful about how we shifted elements. Array shifts are very expensive.

However, in a linked list, the situation is much easier. Rather than shifting and swapping elements, we can actually create two different linked lists: one for elements less than x , and one for elements greater than or equal to x .

We iterate through the linked list, inserting elements into our before list or our after list. Once we reach the end of the linked list and have completed this splitting, we merge the two lists.

This approach is mostly "stable" in that elements stay in their original order, other than the necessary movement around the partition. The code below implements this approach.

```
1  /* Pass in the head of the linked list and the value to partition around */
2  LinkedListNode partition(LinkedListNode node, int x) {
3      LinkedListNode beforeStart = null;
4      LinkedListNode beforeEnd = null;
5      LinkedListNode afterStart = null;
6      LinkedListNode afterEnd = null;
7
8      /* Partition list */
9      while (node != null) {
10         LinkedListNode next = node.next;
11         node.next = null;
12         if (node.data < x) {
13             /* Insert node into end of before list */
14             if (beforeStart == null) {
15                 beforeStart = node;
16                 beforeEnd = beforeStart;
17             } else {
18                 beforeEnd.next = node;
19                 beforeEnd = node;
20             }
21         } else {
22             /* Insert node into end of after list */
23             if (afterStart == null) {
24                 afterStart = node;
25                 afterEnd = afterStart;
26             } else {
```

```

27         afterEnd.next = node;
28         afterEnd = node;
29     }
30 }
31 node = next;
32 }
33
34 if (beforeStart == null) {
35     return afterStart;
36 }
37
38 /* Merge before list and after list */
39 beforeEnd.next = afterStart;
40 return beforeStart;
41 }

```

If it bugs you to keep around four different variables for tracking two linked lists, you're not alone. We can make this code a bit shorter.

If we don't care about making the elements of the list "stable" (which there's no obligation to, since the interviewer hasn't specified that), then we can instead rearrange the elements by growing the list at the head and tail.

In this approach, we start a "new" list (using the existing nodes). Elements bigger than the pivot element are put at the tail and elements smaller are put at the head. Each time we insert an element, we update either the head or tail.

```

1  LinkedListNode partition(LinkedListNode node, int x) {
2      LinkedListNode head = node;
3      LinkedListNode tail = node;
4
5      while (node != null) {
6          LinkedListNode next = node.next;
7          if (node.data < x) {
8              /* Insert node at head. */
9              node.next = head;
10             head = node;
11         } else {
12             /* Insert node at tail. */
13             tail.next = node;
14             tail = node;
15         }
16         node = next;
17     }
18     tail.next = null;
19
20     // The head has changed, so we need to return it to the user.
21     return head;
22 }

```

There are many equally optimal solutions to this problem. If you came up with a different one, that's okay!

- 2.5 Sum Lists:** You have two numbers represented by a linked list, where each node contains a single digit. The digits are stored in reverse order, such that the 1's digit is at the head of the list. Write a function that adds the two numbers and returns the sum as a linked list.

EXAMPLE

Input: (7 -> 1 -> 6) + (5 -> 9 -> 2). That is, 617 + 295.

Output: 2 -> 1 -> 9. That is, 912.

FOLLOW UP

Suppose the digits are stored in forward order. Repeat the above problem.

Input: (6 -> 1 -> 7) + (2 -> 9 -> 5). That is, 617 + 295.

Output: 9 -> 1 -> 2. That is, 912.

pg 95

SOLUTION

It's useful to remember in this problem how exactly addition works. Imagine the problem:

```
  6 1 7
+ 2 9 5
```

First, we add 7 and 5 to get 12. The digit 2 becomes the last digit of the number, and 1 gets carried over to the next step. Second, we add 1, 1, and 9 to get 11. The 1 becomes the second digit, and the other 1 gets carried over the final step. Third and finally, we add 1, 6 and 2 to get 9. So, our value becomes 912.

We can mimic this process recursively by adding node by node, carrying over any "excess" data to the next node. Let's walk through this for the below linked list:

```
  7 -> 1 -> 6
+ 5 -> 9 -> 2
```

We do the following:

1. We add 7 and 5 first, getting a result of 12. 2 becomes the first node in our linked list, and we "carry" the 1 to the next sum.

List: 2 -> ?

2. We then add 1 and 9, as well as the "carry," getting a result of 11. 1 becomes the second element of our linked list, and we carry the 1 to the next sum.

List: 2 -> 1 -> ?

3. Finally, we add 6, 2 and our "carry," to get 9. This becomes the final element of our linked list.

List: 2 -> 1 -> 9.

The code below implements this algorithm.

```
1  LinkedListNode addLists(LinkedListNode l1, LinkedListNode l2, int carry) {
2      if (l1 == null && l2 == null && carry == 0) {
3          return null;
4      }
5
6      LinkedListNode result = new LinkedListNode();
7      int value = carry;
8      if (l1 != null) {
9          value += l1.data;
10     }
11     if (l2 != null) {
```



```

12     value += l2.data;
13 }
14
15 result.data = value % 10; /* Second digit of number */
16
17 /* Recurse */
18 if (l1 != null || l2 != null) {
19     LinkedListNode more = addLists(l1 == null ? null : l1.next,
20                                   l2 == null ? null : l2.next,
21                                   value >= 10 ? 1 : 0);
22     result.setNext(more);
23 }
24 return result;
25 }

```

In implementing this code, we must be careful to handle the condition when one linked list is shorter than another. We don't want to get a null pointer exception.

Follow Up

Part B is conceptually the same (recurse, carry the excess), but has some additional complications when it comes to implementation:

1. One list may be shorter than the other, and we cannot handle this "on the fly." For example, suppose we were adding (1 -> 2 -> 3 -> 4) and (5 -> 6 -> 7). We need to know that the 5 should be "matched" with the 2, not the 1. We can accomplish this by comparing the lengths of the lists in the beginning and padding the shorter list with zeros.
2. In the first part, successive results were added to the tail (i.e., passed forward). This meant that the recursive call would be *passed* the carry, and would return the result (which is then appended to the tail). In this case, however, results are added to the head (i.e., passed backward). The recursive call must return the result, as before, as well as the carry. This is not terribly challenging to implement, but it is more cumbersome. We can solve this issue by creating a wrapper class called Partial Sum.

The code below implements this algorithm.

```

1  class PartialSum {
2      public LinkedListNode sum = null;
3      public int carry = 0;
4  }
5
6  LinkedListNode addLists(LinkedListNode l1, LinkedListNode l2) {
7      int len1 = length(l1);
8      int len2 = length(l2);
9
10     /* Pad the shorter list with zeros - see note (1) */
11     if (len1 < len2) {
12         l1 = padList(l1, len2 - len1);
13     } else {
14         l2 = padList(l2, len1 - len2);
15     }
16
17     /* Add lists */
18     PartialSum sum = addListsHelper(l1, l2);
19
20     /* If there was a carry value left over, insert this at the front of the list.
21      * Otherwise, just return the linked list. */
22     if (sum.carry == 0) {

```

```
23     return sum.sum;
24 } else {
25     LinkedListNode result = insertBefore(sum.sum, sum.carry);
26     return result;
27 }
28 }
29
30 PartialSum addListsHelper(LinkedListNode l1, LinkedListNode l2) {
31     if (l1 == null && l2 == null) {
32         PartialSum sum = new PartialSum();
33         return sum;
34     }
35     /* Add smaller digits recursively */
36     PartialSum sum = addListsHelper(l1.next, l2.next);
37
38     /* Add carry to current data */
39     int val = sum.carry + l1.data + l2.data;
40
41     /* Insert sum of current digits */
42     LinkedListNode full_result = insertBefore(sum.sum, val % 10);
43
44     /* Return sum so far, and the carry value */
45     sum.sum = full_result;
46     sum.carry = val / 10;
47     return sum;
48 }
49
50 /* Pad the list with zeros */
51 LinkedListNode padList(LinkedListNode l, int padding) {
52     LinkedListNode head = l;
53     for (int i = 0; i < padding; i++) {
54         head = insertBefore(head, 0);
55     }
56     return head;
57 }
58
59 /* Helper function to insert node in the front of a linked list */
60 LinkedListNode insertBefore(LinkedListNode list, int data) {
61     LinkedListNode node = new LinkedListNode(data);
62     if (list != null) {
63         node.next = list;
64     }
65     return node;
66 }
```

Note how we have pulled `insertBefore()`, `padList()`, and `length()` (not listed) into their own methods. This makes the code cleaner and easier to read—a wise thing to do in your interviews!

2.6 Palindrome: Implement a function to check if a linked list is a palindrome.

pg 95

SOLUTION

To approach this problem, we can picture a palindrome like `0 -> 1 -> 2 -> 1 -> 0`. We know that, since it's a palindrome, the list must be the same backwards and forwards. This leads us to our first solution.

Solution #1: Reverse and Compare

Our first solution is to reverse the linked list and compare the reversed list to the original list. If they're the same, the lists are identical.

Note that when we compare the linked list to the reversed list, we only actually need to compare the first half of the list. If the first half of the normal list matches the first half of the reversed list, then the second half of the normal list must match the second half of the reversed list.

```

1  boolean isPalindrome(LinkedListNode head) {
2      LinkedListNode reversed = reverseAndClone(head);
3      return isEqual(head, reversed);
4  }
5
6  LinkedListNode reverseAndClone(LinkedListNode node) {
7      LinkedListNode head = null;
8      while (node != null) {
9          LinkedListNode n = new LinkedListNode(node.data); // Clone
10         n.next = head;
11         head = n;
12         node = node.next;
13     }
14     return head;
15 }
16
17 boolean isEqual(LinkedListNode one, LinkedListNode two) {
18     while (one != null && two != null) {
19         if (one.data != two.data) {
20             return false;
21         }
22         one = one.next;
23         two = two.next;
24     }
25     return one == null && two == null;
26 }

```

Observe that we've modularized this code into `reverse` and `isEqual` functions. We've also created a new class so that we can return both the head and the tail of this method. We could have also returned a two-element array, but that approach is less maintainable.

Solution #2: Iterative Approach

We want to detect linked lists where the front half of the list is the reverse of the second half. How would we do that? By reversing the front half of the list. A stack can accomplish this.

We need to push the first half of the elements onto a stack. We can do this in two different ways, depending on whether or not we know the size of the linked list.

If we know the size of the linked list, we can iterate through the first half of the elements in a standard for loop, pushing each element onto a stack. We must be careful, of course, to handle the case where the length of the linked list is odd.

If we don't know the size of the linked list, we can iterate through the linked list, using the fast runner / slow runner technique described in the beginning of the chapter. At each step in the loop, we push the data from the slow runner onto a stack. When the fast runner hits the end of the list, the slow runner will have reached the middle of the linked list. By this point, the stack will have all the elements from the front of the linked list, but in reverse order.

Now, we simply iterate through the rest of the linked list. At each iteration, we compare the node to the top of the stack. If we complete the iteration without finding a difference, then the linked list is a palindrome.

```
1  boolean isPalindrome(ListNode head) {
2      ListNode fast = head;
3      ListNode slow = head;
4
5      Stack<Integer> stack = new Stack<Integer>();
6
7      /* Push elements from first half of linked list onto stack. When fast runner
8       * (which is moving at 2x speed) reaches the end of the linked list, then we
9       * know we're at the middle */
10     while (fast != null && fast.next != null) {
11         stack.push(slow.data);
12         slow = slow.next;
13         fast = fast.next.next;
14     }
15
16     /* Has odd number of elements, so skip the middle element */
17     if (fast != null) {
18         slow = slow.next;
19     }
20
21     while (slow != null) {
22         int top = stack.pop().intValue();
23
24         /* If values are different, then it's not a palindrome */
25         if (top != slow.data) {
26             return false;
27         }
28         slow = slow.next;
29     }
30     return true;
31 }
```

Solution #3: Recursive Approach

First, a word on notation: in this solution, when we use the notation node Kx, the variable K indicates the value of the node data, and x (which is either f or b) indicates whether we are referring to the front node with that value or the back node. For example, in the below linked list, node 2b would refer to the second (back) node with value 2.

Now, like many linked list problems, you can approach this problem recursively. We may have some intuitive idea that we want to compare element 0 and element $n - 1$, element 1 and element $n - 2$, element 2 and element $n - 3$, and so on, until the middle element(s). For example:

0 (1 (2 (3) 2) 1) 0

In order to apply this approach, we first need to know when we've reached the middle element, as this will form our base case. We can do this by passing in $\text{length} - 2$ for the `length` each time. When the `length` equals 0 or 1, we're at the center of the linked list. This is because the `length` is reduced by 2 each time. Once we've recursed $\frac{n}{2}$ times, `length` will be down to 0.

```
1  recurse(Node n, int length) {
2      if (length == 0 || length == 1) {
3          return [something]; // At middle
4      }
5      recurse(n.next, length - 2);
```

```

6     ...
7 }

```

This method will form the outline of the `isPalindrome` method. The “meat” of the algorithm though is comparing node `i` to node `n - i` to check if the linked list is a palindrome. How do we do that?

Let’s examine what the call stack looks like:

```

1  v1 = isPalindrome: list = 0 ( 1 ( 2 ( 3 ) 2 ) 1 ) 0. length = 7
2    v2 = isPalindrome: list = 1 ( 2 ( 3 ) 2 ) 1 ) 0. length = 5
3      v3 = isPalindrome: list = 2 ( 3 ) 2 ) 1 ) 0. length = 3
4        v4 = isPalindrome: list = 3 ) 2 ) 1 ) 0. length = 1
5          returns v3
6        returns v2
7      returns v1
8    returns ?

```

In the above call stack, each call wants to check if the list is a palindrome by comparing its head node with the corresponding node from the back of the list. That is:

- Line 1 needs to compare node `0f` with node `0b`
- Line 2 needs to compare node `1f` with node `1b`
- Line 3 needs to compare node `2f` with node `2b`
- Line 4 needs to compare node `3f` with node `3b`.

If we rewind the stack, passing nodes back as described below, we can do just that:

- Line 4 sees that it is the middle node (since `length = 1`), and passes back `head.next`. The value `head` equals node `3`, so `head.next` is node `2b`.
- Line 3 compares its head, node `2f`, to `returned_node` (the value from the previous recursive call), which is node `2b`. If the values match, it passes a reference to node `1b` (`returned_node.next`) up to line 2.
- Line 2 compares its head (node `1f`) to `returned_node` (node `1b`). If the values match, it passes a reference to node `0b` (or, `returned_node.next`) up to line 1.
- Line 1 compares its head, node `0f`, to `returned_node`, which is node `0b`. If the values match, it returns `true`.

To generalize, each call compares its head to `returned_node`, and then passes `returned_node.next` up the stack. In this way, every node `i` gets compared to node `n - i`. If at any point the values do not match, we return `false`, and every call up the stack checks for that value.

But wait, you might ask, sometimes we said we’ll return a `boolean` value, and sometimes we’re returning a node. Which is it?

It’s both. We create a simple class with two members, a `boolean` and a `node`, and return an instance of that class.

```

1 class Result {
2     public LinkedListNode node;
3     public boolean result;
4 }

```

The example below illustrates the parameters and return values from this sample list.

```

1  isPalindrome: list = 0 ( 1 ( 2 ( 3 ( 4 ) 3 ) 2 ) 1 ) 0. len = 9
2    isPalindrome: list = 1 ( 2 ( 3 ( 4 ) 3 ) 2 ) 1 ) 0. len = 7
3      isPalindrome: list = 2 ( 3 ( 4 ) 3 ) 2 ) 1 ) 0. len = 5

```



```
4         isPalindrome: list = 3 ( 4 ) 3 ) 2 ) 1 ) 0. len = 3
5         isPalindrome: list = 4 ) 3 ) 2 ) 1 ) 0. len = 1
6         returns node 3b, true
7         returns node 2b, true
8         returns node 1b, true
9         returns node 0b, true
10        returns null, true
```

Implementing this code is now just a matter of filling in the details.

```
1  boolean isPalindrome(LinkedListNode head) {
2      int length = lengthOfList(head);
3      Result p = isPalindromeRecurse(head, length);
4      return p.result;
5  }
6
7  Result isPalindromeRecurse(LinkedListNode head, int length) {
8      if (head == null || length <= 0) { // Even number of nodes
9          return new Result(head, true);
10     } else if (length == 1) { // Odd number of nodes
11         return new Result(head.next, true);
12     }
13
14     /* Recurse on sublist. */
15     Result res = isPalindromeRecurse(head.next, length - 2);
16
17     /* If child calls are not a palindrome, pass back up
18      * a failure. */
19     if (!res.result || res.node == null) {
20         return res;
21     }
22
23     /* Check if matches corresponding node on other side. */
24     res.result = (head.data == res.node.data);
25
26     /* Return corresponding node. */
27     res.node = res.node.next;
28
29     return res;
30 }
31
32 int lengthOfList(LinkedListNode n) {
33     int size = 0;
34     while (n != null) {
35         size++;
36         n = n.next;
37     }
38     return size;
39 }
```

Some of you might be wondering why we went through all this effort to create a special `Result` class. Isn't there a better way? Not really—at least not in Java.

However, if we were implementing this in C or C++, we could have passed in a double pointer.

```
1  bool isPalindromeRecurse(Node head, int length, Node** next) {
2      ...
3  }
```

It's ugly, but it works.

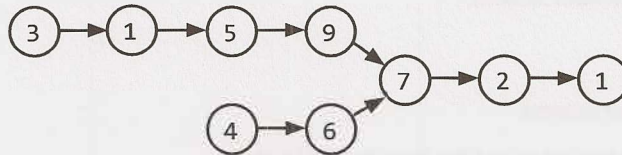
2.7 Intersection: Given two (singly) linked lists, determine if the two lists intersect. Return the intersecting node. Note that the intersection is defined based on reference, not value. That is, if the k th node of the first linked list is the exact same node (by reference) as the j th node of the second linked list, then they are intersecting.

pg 95

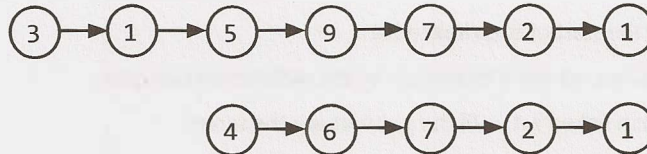
SOLUTION

Let's draw a picture of intersecting linked lists to get a better feel for what is going on.

Here is a picture of intersecting linked lists:



And here is a picture of non-intersecting linked lists:



We should be careful here to not inadvertently draw a special case by making the linked lists the same length.

Let's first ask how we would determine if two linked lists intersect.

Determining if there's an intersection.

How would we detect if two linked lists intersect? One approach would be to use a hash table and just throw all the linked lists nodes into there. We would need to be careful to reference the linked lists by their memory location, not by their value.

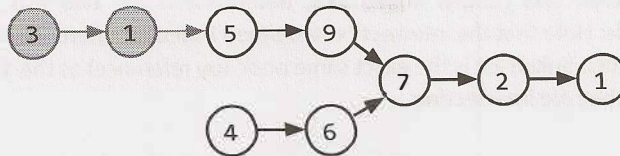
There's an easier way though. Observe that two intersecting linked lists will always have the same last node. Therefore, we can just traverse to the end of each linked list and compare the last nodes.

How do we find where the intersection is, though?

Finding the intersecting node.

One thought is that we could traverse backwards through each linked list. When the linked lists "split", that's the intersection. Of course, you can't really traverse backwards through a singly linked list.

If the linked lists were the same length, you could just traverse through them at the same time. When they collide, that's your intersection.



When they're not the same length, we'd like to just "chop off"—or ignore—those excess (gray) nodes.

How can we do this? Well, if we know the lengths of the two linked lists, then the difference between those two linked lists will tell us how much to chop off.

We can get the lengths at the same time as we get the tails of the linked lists (which we used in the first step to determine if there's an intersection).

Putting it all together.

We now have a multistep process.

1. Run through each linked list to get the lengths and the tails.
2. Compare the tails. If they are different (by reference, not by value), return immediately. There is no intersection.
3. Set two pointers to the start of each linked list.
4. On the longer linked list, advance its pointer by the difference in lengths.
5. Now, traverse on each linked list until the pointers are the same.

The implementation for this is below.

```

1  LinkedListNode findIntersection(LinkedListNode list1, LinkedListNode list2) {
2      if (list1 == null || list2 == null) return null;
3
4      /* Get tail and sizes. */
5      Result result1 = getTailAndSize(list1);
6      Result result2 = getTailAndSize(list2);
7
8      /* If different tail nodes, then there's no intersection. */
9      if (result1.tail != result2.tail) {
10         return null;
11     }
12
13     /* Set pointers to the start of each linked list. */
14     LinkedListNode shorter = result1.size < result2.size ? list1 : list2;
15     LinkedListNode longer = result1.size > result2.size ? list1 : list2;
16
17     /* Advance the pointer for the longer linked list by difference in lengths. */
18     longer = getKthNode(longer, Math.abs(result1.size - result2.size));
19
20     /* Move both pointers until you have a collision. */
21     while (shorter != longer) {
22         shorter = shorter.next;
23         longer = longer.next;
24     }
25
26     /* Return either one. */
27     return longer;
28 }
29

```

```

30 class Result {
31     public LinkedListNode tail;
32     public int size;
33     public Result(LinkedListNode tail, int size) {
34         this.tail = tail;
35         this.size = size;
36     }
37 }
38
39 Result getTailAndSize(LinkedListNode list) {
40     if (list == null) return null;
41
42     int size = 1;
43     LinkedListNode current = list;
44     while (current.next != null) {
45         size++;
46         current = current.next;
47     }
48     return new Result(current, size);
49 }
50
51 LinkedListNode getKthNode(LinkedListNode head, int k) {
52     LinkedListNode current = head;
53     while (k > 0 && current != null) {
54         current = current.next;
55         k--;
56     }
57     return current;
58 }

```

This algorithm takes $O(A + B)$ time, where A and B are the lengths of the two linked lists. It takes $O(1)$ additional space.

2.8 Loop Detection: Given a circular linked list, implement an algorithm that returns the node at the beginning of the loop.

DEFINITION

Circular linked list: A (corrupt) linked list in which a node's next pointer points to an earlier node, so as to make a loop in the linked list.

EXAMPLE

Input: A -> B -> C -> D -> E -> C [the same C as earlier]

Output: C

pg 95

SOLUTION

This is a modification of a classic interview problem: detect if a linked list has a loop. Let's apply the Pattern Matching approach.

Part 1: Detect If Linked List Has A Loop

An easy way to detect if a linked list has a loop is through the FastRunner / SlowRunner approach. FastRunner moves two steps at a time, while SlowRunner moves one step. Much like two cars racing around a track at different steps, they must eventually meet.

An astute reader may wonder if FastRunner might “hop over” SlowRunner completely, without ever colliding. That’s not possible. Suppose that FastRunner *did* hop over SlowRunner, such that SlowRunner is at spot i and FastRunner is at spot $i + 1$. In the previous step, SlowRunner would be at spot $i - 1$ and FastRunner would be at spot $((i + 1) - 2)$, or spot $i - 1$. That is, they would have collided.

Part 2: When Do They Collide?

Let’s assume that the linked list has a “non-looped” part of size k .

If we apply our algorithm from part 1, when will FastRunner and SlowRunner collide?

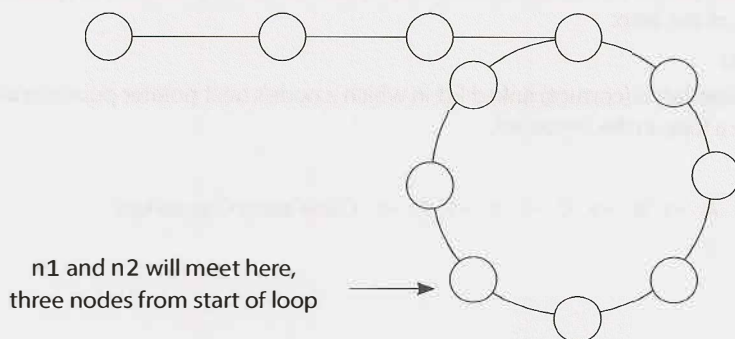
We know that for every p steps that SlowRunner takes, FastRunner has taken $2p$ steps. Therefore, when SlowRunner enters the looped portion after k steps, FastRunner has taken $2k$ steps total and must be $2k - k$ steps, or k steps, into the looped portion. Since k might be much larger than the loop length, we should actually write this as $\text{mod}(k, \text{LOOP_SIZE})$ steps, which we will denote as K .

At each subsequent step, FastRunner and SlowRunner get either one step farther away or one step closer, depending on your perspective. That is, because we are in a circle, when A moves q steps away from B, it is also moving q steps closer to B.

So now we know the following facts:

1. SlowRunner is 0 steps into the loop.
2. FastRunner is K steps into the loop.
3. SlowRunner is K steps behind FastRunner.
4. FastRunner is $\text{LOOP_SIZE} - K$ steps behind SlowRunner.
5. FastRunner catches up to SlowRunner at a rate of 1 step per unit of time.

So, when do they meet? Well, if FastRunner is $\text{LOOP_SIZE} - K$ steps behind SlowRunner, and FastRunner catches up at a rate of 1 step per unit of time, then they meet after $\text{LOOP_SIZE} - K$ steps. At this point, they will be K steps before the head of the loop. Let’s call this point CollisionSpot.



Part 3: How Do You Find The Start of the Loop?

We now know that CollisionSpot is K nodes before the start of the loop. Because $K = \text{mod}(k, \text{LOOP_SIZE})$ (or, in other words, $k = K + M * \text{LOOP_SIZE}$, for any integer M), it is also correct to say that it is k nodes from the loop start. For example, if node N is 2 nodes into a 5 node loop, it is also correct to say that it is 7, 12, or even 397 nodes into the loop.

Therefore, both CollisionSpot and LinkedListHead are k nodes from the start of the loop.

Now, if we keep one pointer at `CollisionSpot` and move the other one to `LinkedListHead`, they will each be k nodes from `LoopStart`. Moving the two pointers at the same speed will cause them to collide again—this time after k steps, at which point they will both be at `LoopStart`. All we have to do is return this node.

Part 4: Putting It All Together

To summarize, we move `FastPointer` twice as fast as `SlowPointer`. When `SlowPointer` enters the loop, after k nodes, `FastPointer` is k nodes into the loop. This means that `FastPointer` and `SlowPointer` are $\text{LOOP_SIZE} - k$ nodes away from each other.

Next, if `FastPointer` moves two nodes for each node that `SlowPointer` moves, they move one node closer to each other on each turn. Therefore, they will meet after $\text{LOOP_SIZE} - k$ turns. Both will be k nodes from the front of the loop.

The head of the linked list is also k nodes from the front of the loop. So, if we keep one pointer where it is, and move the other pointer to the head of the linked list, then they will meet at the front of the loop.

Our algorithm is derived directly from parts 1, 2 and 3.

1. Create two pointers, `FastPointer` and `SlowPointer`.
2. Move `FastPointer` at a rate of 2 steps and `SlowPointer` at a rate of 1 step.
3. When they collide, move `SlowPointer` to `LinkedListHead`. Keep `FastPointer` where it is.
4. Move `SlowPointer` and `FastPointer` at a rate of one step. Return the new collision point.

The code below implements this algorithm.

```

1  LinkedListNode FindBeginning(LinkedListNode head) {
2      LinkedListNode slow = head;
3      LinkedListNode fast = head;
4
5      /* Find meeting point. This will be LOOP_SIZE - k steps into the linked list. */
6      while (fast != null && fast.next != null) {
7          slow = slow.next;
8          fast = fast.next.next;
9          if (slow == fast) { // Collision
10             break;
11         }
12     }
13
14     /* Error check - no meeting point, and therefore no loop */
15     if (fast == null || fast.next == null) {
16         return null;
17     }
18
19     /* Move slow to Head. Keep fast at Meeting Point. Each are k steps from the
20      * Loop Start. If they move at the same pace, they must meet at Loop Start. */
21     slow = head;
22     while (slow != fast) {
23         slow = slow.next;
24         fast = fast.next;
25     }
26
27     /* Both now point to the start of the loop. */
28     return fast;
29 }
```


3

Solutions to Stacks and Queues

3.1 Three in One: Describe how you could use a single array to implement three stacks.

pg 98

SOLUTION

Like many problems, this one somewhat depends on how well we'd like to support these stacks. If we're okay with simply allocating a fixed amount of space for each stack, we can do that. This may mean though that one stack runs out of space, while the others are nearly empty.

Alternatively, we can be flexible in our space allocation, but this significantly increases the complexity of the problem.

Approach 1: Fixed Division

We can divide the array in three equal parts and allow the individual stack to grow in that limited space. Note: We will use the notation "[" to mean inclusive of an end point and "(" to mean exclusive of an end point.

- For stack 1, we will use $[0, \frac{n}{3})$.
- For stack 2, we will use $[\frac{n}{3}, \frac{2n}{3})$.
- For stack 3, we will use $[\frac{2n}{3}, n)$.

The code for this solution is below.

```
1 class FixedMultiStack {
2     private int numberOfStacks = 3;
3     private int stackCapacity;
4     private int[] values;
5     private int[] sizes;
6
7     public FixedMultiStack(int stackSize) {
8         stackCapacity = stackSize;
9         values = new int[stackSize * numberOfStacks];
10        sizes = new int[numberOfStacks];
11    }
12
13    /* Push value onto stack. */
14    public void push(int stackNum, int value) throws FullStackException {
15        /* Check that we have space for the next element */
16        if (isFull(stackNum)) {
17            throw new FullStackException();
18        }
19    }
20
21    /* Pop value from stack. */
22    public int pop(int stackNum) throws EmptyStackException {
23        if (isEmpty(stackNum)) {
24            throw new EmptyStackException();
25        }
26        sizes[stackNum]--;
27        return values[stackNum * stackCapacity + sizes[stackNum]];
28    }
29
30    /* Return true if the stack is full, false otherwise. */
31    public boolean isFull(int stackNum) {
32        return sizes[stackNum] == stackCapacity;
33    }
34
35    /* Return true if the stack is empty, false otherwise. */
36    public boolean isEmpty(int stackNum) {
37        return sizes[stackNum] == 0;
38    }
39}
```

```
18     }
19
20     /* Increment stack pointer and then update top value. */
21     sizes[stackNum]++;
22     values[indexOfTop(stackNum)] = value;
23 }
24
25 /* Pop item from top stack. */
26 public int pop(int stackNum) {
27     if (isEmpty(stackNum)) {
28         throw new EmptyStackException();
29     }
30
31     int topIndex = indexOfTop(stackNum);
32     int value = values[topIndex]; // Get top
33     values[topIndex] = 0; // Clear
34     sizes[stackNum]--; // Shrink
35     return value;
36 }
37
38 /* Return top element. */
39 public int peek(int stackNum) {
40     if (isEmpty(stackNum)) {
41         throw new EmptyStackException();
42     }
43     return values[indexOfTop(stackNum)];
44 }
45
46 /* Return if stack is empty. */
47 public boolean isEmpty(int stackNum) {
48     return sizes[stackNum] == 0;
49 }
50
51 /* Return if stack is full. */
52 public boolean isFull(int stackNum) {
53     return sizes[stackNum] == stackCapacity;
54 }
55
56 /* Returns index of the top of the stack. */
57 private int indexOfTop(int stackNum) {
58     int offset = stackNum * stackCapacity;
59     int size = sizes[stackNum];
60     return offset + size - 1;
61 }
62 }
```

If we had additional information about the expected usages of the stacks, then we could modify this algorithm accordingly. For example, if we expected Stack 1 to have many more elements than Stack 2, we could allocate more space to Stack 1 and less space to Stack 2.

Approach 2: Flexible Divisions

A second approach is to allow the stack blocks to be flexible in size. When one stack exceeds its initial capacity, we grow the allowable capacity and shift elements as necessary.

We will also design our array to be circular, such that the final stack may start at the end of the array and wrap around to the beginning.

Please note that the code for this solution is far more complex than would be appropriate for an interview. You could be responsible for pseudocode, or perhaps the code of individual components, but the entire implementation would be far too much work.

```

1 public class MultiStack {
2     /* StackInfo is a simple class that holds a set of data about each stack. It
3      * does not hold the actual items in the stack. We could have done this with
4      * just a bunch of individual variables, but that's messy and doesn't gain us
5      * much. */
6     private class StackInfo {
7         public int start, size, capacity;
8         public StackInfo(int start, int capacity) {
9             this.start = start;
10            this.capacity = capacity;
11        }
12
13        /* Check if an index on the full array is within the stack boundaries. The
14         * stack can wrap around to the start of the array. */
15        public boolean isWithinStackCapacity(int index) {
16            /* If outside of bounds of array, return false. */
17            if (index < 0 || index >= values.length) {
18                return false;
19            }
20
21            /* If index wraps around, adjust it. */
22            int contiguousIndex = index < start ? index + values.length : index;
23            int end = start + capacity;
24            return start <= contiguousIndex && contiguousIndex < end;
25        }
26
27        public int lastCapacityIndex() {
28            return adjustIndex(start + capacity - 1);
29        }
30
31        public int lastElementIndex() {
32            return adjustIndex(start + size - 1);
33        }
34
35        public boolean isFull() { return size == capacity; }
36        public boolean isEmpty() { return size == 0; }
37    }
38
39    private StackInfo[] info;
40    private int[] values;
41
42    public MultiStack(int numberOfStacks, int defaultSize) {
43        /* Create metadata for all the stacks. */
44        info = new StackInfo[numberOfStacks];
45        for (int i = 0; i < numberOfStacks; i++) {
46            info[i] = new StackInfo(defaultSize * i, defaultSize);
47        }
48        values = new int[numberOfStacks * defaultSize];
49    }
50
51    /* Push value onto stack num, shifting/expanding stacks as necessary. Throws
52     * exception if all stacks are full. */
53    public void push(int stackNum, int value) throws FullStackException {

```